An Analysis of Bagging ensemble

Bagging ensemble

Bagging ensemble -- Part 1

For a weak learner with high bias, bagging will also carry high bias into its aggregate Loss of interpretability of a model. Can be computationally expensive depending on the dataset.

Bagging ensemble -- Part 2

This process is repeated recursively for successive levels of the tree until the desired depth is reached. At the very bottom of the tree, samples that test positive for the final feature are generally classified as positive, while those that lack the feature are classified as negative. These trees are then used as predictors to classify new data.

Bagging ensemble -- Part 3

Create m {\displaystyle m} new training sets D i {\displaystyle D_{i}} , from D {\displaystyle D} with replacement Classifier C i {\displaystyle C_{i}} is built from each set D i {\displaystyle D_{i}} using I {\displaystyle I} to determine the classification of set D i {\displaystyle D_{i}}

Bagging ensemble -- Part 4

Specify the maximum depth of trees: Instead of allowing the random forest to continue until all nodes are pure, it is better to cut it off at a certain point in order to further decrease chances of overfitting. Prune the dataset: Using an extremely large dataset may create results that are less indicative of the data provided than a smaller set that more accurately represents what is being focused on. Continue pruning the data at each node split rather than just in the original bagging process. Decide on accuracy or speed: Depending on the desired results, increasing or decreasing the number of trees within the forest can help. Increasing the number of trees generally provides more accurate results while decreasing the number of trees will provide quicker results.

Bagging ensemble -- Part 5

Many weak learners aggregated typically outperform a single learner over the entire set, and have less overfit Reduces variance in high-variance low-bias weak learner, which can improve efficiency (statistics) Can be performed in parallel, as each separate bootstrap can be processed on its own before aggregation. Disadvantages:

Bagging ensemble -- Part 6

Creating the bootstrap dataset The bootstrap dataset is made by randomly picking objects from the original dataset. Also, it must be the same size as the original dataset. However, the difference is that the bootstrap dataset can have duplicate objects. Here is a simple example to demonstrate how it works along with the illustration below:

Bagging ensemble -- Part 7

Suppose the original dataset is a group of 12 people. Their names are Emily, Jessie, George, Constantine, Lexi, Theodore, John, James, Rachel, Anthony, Ellie, and Jamal. By randomly picking a group of names, let us say our bootstrap dataset had James, Ellie, Constantine, Lexi, John, Constantine, Theodore, Constantine, Anthony, Lexi, Constantine, and Theodore. In this case, the bootstrap sample contained four duplicates for Constantine, and two duplicates for Lexi, and Theodore.

Bagging ensemble -- Part 8

Creating the out-of-bag dataset The out-of-bag dataset represents the remaining people who were not in the bootstrap dataset. It can be calculated by taking the difference between the original and the bootstrap datasets. In this case, the remaining samples who were not selected are Emily, Jessie, George, Rachel, and Jamal. Keep in mind that since both datasets are sets, when taking the difference the duplicate names are ignored in the bootstrap dataset. The illustration below shows how the math is done:

Bagging ensemble -- Part 9

Random Forests The next part of the algorithm involves introducing yet another element of variability amongst the bootstrapped trees. In addition to each tree only examining a bootstrapped set of samples, only a small but consistent number of unique features are considered when ranking them as classifiers. This means that each tree only knows about the data pertaining to a small constant number of features, and a variable number of samples that is less than or equal to that of the original dataset. Consequently, the trees are more likely to return a wider array of answers, derived from more diverse knowledge. This results in a random forest, which possesses numerous benefits over a single decision tree generated without randomness. In a random forest, each tree "votes" on whether or not to classify a sample as positive based on its features. The sample is then classified based on majority vote. An example of this is given in the diagram below, where the four trees in a random forest vote on whether or not a patient with mutations A, B, F, and G has cancer. Since three out of four trees vote yes, the patient is then classified as cancer positive.

Bagging ensemble -- Part 10

Key Terms There are three types of datasets in bootstrap aggregating. These are the original, bootstrap, and out-of-bag datasets. Each section below will explain how each dataset is made except for the original dataset. The original dataset is whatever information is given.

Bagging ensemble -- Part 11

Improving Random Forests and Bagging While the techniques described above utilize random forests and bagging (otherwise known as bootstrapping), there are certain techniques that can be used in order to improve their execution and voting time, their prediction accuracy, and their overall performance. The following are key steps in creating an efficient random forest:

Bagging ensemble -- Part 12

Finally classifier C ∗ {\displaystyle C^{*}} is generated by using the previously created set of classifiers C i {\displaystyle C_{i}} on the original dataset D {\displaystyle D} , the classification predicted most often by the sub-classifiers C i {\displaystyle C_{i}} is the final classification for i = 1 to m { D' = bootstrap sample from D (sample with replacement) Ci = I(D') } C*(x) = argmax #{i:Ci(x)=y} (most often predicted label y) y∈ Y

Bagging ensemble -- Part 13

Description of the technique Given a standard training set D {\displaystyle D} of size n {\displaystyle n} , bagging generates m {\displaystyle m} new training sets D i {\displaystyle D_{i}} , each of size n ′ {\displaystyle n'} , by sampling from D {\displaystyle D} uniformly and with replacement. By sampling with replacement, some observations may be repeated in each D i {\displaystyle D_{i}} . If n ′ = n {\displaystyle n'=n} , then for large n {\displaystyle n} the set D i {\displaystyle D_{i}} is expected to have the fraction (1 - 1/e) (~63.2%) of the unique samples of D {\displaystyle D} , the rest being duplicates. This kind of sample is known as a bootstrap sample. Sampling with replacement ensures each bootstrap is independent from its peers, as it does not depend on previous chosen samples when sampling. Then, m {\displaystyle m} models are fitted using the above bootstrap samples and combined by averaging the output (for regression) or voting (for classification).